Sandbox Overhead and Defense-in-Depth, Worked Out by Hand

Why three weak layers beat one strong one — and what a warm sandbox pool really buys you — every probability and second counted.

What this document does. It takes the one defensive picture from Module 4.6 — an agent that runs untrusted code, where a prompt injection must pass an *input filter*, a *sandbox*, and a *network policy* before it can do harm — and computes two things by hand: the probability a breach survives all the layers, and the latency a sandbox adds to each tool call. Nothing is hand-waved. Each number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. We use the module's own figures: E2B-style cold starts of ∼100 ms against an ephemeral ∼1 s, and the five-layer stack (input filter · sandbox · scopes · runtime monitor · audit) whose "probability of total bypass drops exponentially with the depth of the stack."

Where this sits. This is **Topic 1 of Module 4.6** (Agent Security & Sandboxing) — the quantitative backbone of the eight-threat / five-layer lesson. Its companions: Module 4.1 (thread auth, which supplies the replay defense) and Tracks 3.3 / 3.4 (the safety pipeline and cost engineering, where the same independence-multiplies idea recurs).

Concepts covered, in order: defense-in-depth as multiplied failure probabilities · $P(\text{breach}) = \prod_i p_i$ · why one layer at p=0.1 leaves 10% · diminishing *absolute* but constant *relative* gains · sandbox cold-start latency · ephemeral vs. warm-pool overhead · the break-even pool size · blast radius as risk = $P(\text{breach}) \times$ impact · defense lowers P, scoping caps impact.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one injection, three gates to clear

An agent that executes code has a much bigger blast radius than a chatbot: the same prompt-injection trick that gives a RAG system a wrong answer can, in an agent, run code, move money, or hit an internal API. Module 4.6 answers this not with one perfect control but with a *stack* of independent ones. Model the attacker's job concretely: a hostile instruction must defeat every layer in series to land. For an agent that runs code we focus on three:

Layer	What it is	Fails to stop with prob.
L_1	input filter (injection signatures, delimiters)	$p_1 = 0.10$
L_2	sandbox isolation (microVM / gVisor jail)	$p_2 = 0.10$
L_3	network policy (egress allowlist, no metadata)	$p_3 = 0.10$

The quantities. Each p_i is the probability that layer i, *on its own*, lets the attack through. We pick $p_i = 0.10$ — a deliberately *weak* layer, blocking only 9 attacks in 10 — so the arithmetic is followable and the conclusion is conservative. For the latency half we use a cold-start time per tool call: $c_{\text{cold}} = 1.0$ s for an ephemeral sandbox spun up per call, $c_{\text{warm}} = 0.1$ s for a pre-warmed pool member, and $N = 5$ tool calls in one trajectory.

Notation. $P(\text{breach})$ is the probability the attack survives *all* layers. With L identical layers it is p^L. The layers must be *independent* — each catches a different signal — or the product rule does not hold. We carry probabilities to several decimals throughout.

Intuition

Defense-in-depth is not "a wall, but taller." It is several different walls, each with its own holes, stacked so the holes rarely line up. A single layer at p=0.1 is a wall with a 10% gap. Put three such walls in series and the attack must find a gap in all three at once: $0.1 \times 0.1 \times 0.1 = 0.001$. You did not build a better wall; you multiplied three mediocre ones into a strong barrier.

2 Step 1 — why one layer is never enough

Take the input filter alone. It is good — it stops 90% of injections. The breach probability is just its own failure rate:

$$P(\text{breach}) = p_1 = 0.10.$$

One in ten attacks gets through. For an agent that can issue refunds or exfiltrate a customer table, a 10% landing rate is not a security posture; it is a countdown. No realistic single filter reaches $p = 0.001$ on adversarial input — injection signatures are imperfect by nature. The only cheap route to three-nines is to *multiply*, not to perfect one layer.

Mini-example — this operation in isolation

Suppose 1,000 injection attempts hit the agent in a month. A single p=0.1 filter lets $1000 \times 0.10 = 100$ of them through — 100 chances to run code or move money. Add an independent sandbox at p=0.1: of those 100 survivors, only $100 \times 0.10 = 10$ also escape the sandbox. Add a network policy at p=0.1: of those 10, only $10 \times 0.10 = 1$ also beats egress control. The funnel $1000 \to 100 \to 10 \to 1$ *is* the product $0.1^3 = 0.001$, read as a body count.

3 Step 2 — the product rule for independent layers

If an attack must pass L independent layers and layer i independently fails to stop it with probability p_i, the layers' failures are independent events, so the breach — the conjunction of all of them failing — has probability equal to the product:

$$\boxed{P(\text{breach}) = \prod_{i=1}^{L} p_i} \tag{1}$$

For our three equal layers, $P(\text{breach}) = p_1 p_2 p_3 = 0.10 \times 0.10 \times 0.10 = 0.001$. The single layer left 10% on the table; the *product* is what drags us down to 0.1%. Check: $0.10/0.001 = 100$, a hundred-fold reduction bought purely by stacking, with no layer improved.

Sweep the depth L, keeping each layer at $p = 0.10$, so $P(\text{breach}) = 0.10^L$:

Layers L	$P(\text{breach}) = 0.10^L$	rel. reduction vs. 1 layer	abs. drop from prev.
1	0.100000	1×	—
2	0.010000	10×	0.09000
3	0.001000	100×	0.00900
4	0.000100	1,000×	0.00090
5	0.000010	10,000×	0.00009

Sample check, $L = 3$: $0.10^3 = 0.001$; reduction $= 0.10/0.001 = 100\times$; absolute drop from L=2 is $0.01 - 0.001 = 0.009$. ✓ Three weak layers reach the 0.1% the module's stack is built around.

Intuition

Read the last two columns together. The **absolute** gain shrinks each time — going from 2 to 3 layers removes only 0.9 percentage points, and 4 to 5 removes a rounding error. But the **relative** gain is rock-steady: every layer divides the breach probability by ten, forever. Security is sold in orders of magnitude, not percentage points, so the "diminishing" returns are an illusion — each layer is worth exactly as much as the last in the currency that matters.

4 Step 3 — the sandbox tax: ephemeral vs. warm pool

Isolation is not free; the sandbox layer L_2 adds latency. Spinning up a fresh microVM or jailed pod for every tool call pays a *cold start* c_{cold} each time. A *warm pool* keeps sandboxes pre-booted, so a call grabs a ready one and pays only c_{warm}. Over a trajectory of N tool calls the added latency — the *latency tax* — is

$$\text{tax}_{\text{ephemeral}} = N\, c_{\text{cold}}, \qquad \text{tax}_{\text{warm}} = N\, c_{\text{warm}}.$$

Working the module's figures, $c_{\text{cold}} = 1.0\,\text{s}$, $c_{\text{warm}} = 0.1\,\text{s}$, over $N = 5$ calls:

Mode	per-call	over $N = 5$ calls	vs. warm
Ephemeral (cold every call)	1.0 s	$5 \times 1.0 = 5.0\,\text{s}$	10×
Warm pool (pre-booted)	0.1 s	$5 \times 0.1 = 0.5\,\text{s}$	1×
Latency tax saved		**4.5 s**	

Sample check: $5 \times 1.0 = 5.0\,\text{s}$ ephemeral, $5 \times 0.1 = 0.5\,\text{s}$ warm, saving $5.0 - 0.5 = 4.5\,\text{s}$, a $5.0/0.5 = 10\times$ reduction. ✓ The warm pool removes nine-tenths of the isolation tax — the *same* 10× the cold-start gap promised, because the tax is linear in the per-call time.

Watch out

A warm pool is not free either. Keeping K sandboxes booted costs idle compute whether or not traffic arrives, and a pool of size K only serves K concurrent calls warm — the $(K{+}1)$th still cold-starts. The pool is a bet that you will reuse it. So the right question is not "warm or ephemeral?" but "how many calls before the pool's fixed warm-up pays for itself?" — which is the break-even below.

5 Step 4 — the break-even pool size

Amortize a warm pool of size K over a stream of calls. The pool is booted once — K cold starts, a fixed $K\,c_{\text{cold}}$ — and thereafter every one of n calls is served warm at c_{warm}. Against the no-pool baseline of $n\,c_{\text{cold}}$, the pool wins when

$$n\,c_{\text{cold}} \;>\; K\,c_{\text{cold}} + n\,c_{\text{warm}} \quad\Longrightarrow\quad n\,(c_{\text{cold}} - c_{\text{warm}}) \;>\; K\,c_{\text{cold}}.$$

Solving for the break-even call count $n^\star$:

$$\boxed{n^\star \;=\; \frac{K\,c_{\text{cold}}}{c_{\text{cold}} - c_{\text{warm}}}} \tag{2}$$

For $K = 5$, $c_{\text{cold}} = 1.0\,\text{s}$, $c_{\text{warm}} = 0.1\,\text{s}$: $n^\star = \frac{5\times1.0}{1.0-0.1} = \frac{5.0}{0.9} = 5.56$. Past about 5.6 calls, a five-member warm pool beats cold-starting every time. Concretely, over $M = 100$ trajectories of $N = 5$ calls (500 calls):

Strategy	**cold starts**	**total isolation latency**
No pool (cold every call)	500	$500 \times 1.0 = 500.0\,\text{s}$
Warm pool ($K = 5$, reused)	5	$5 \times 1.0 + 500 \times 0.1 = 55.0\,\text{s}$
Saving		**445.0 s** ($\approx 9.1\times$)

Sample check: pool total $= 5\times1.0+500\times0.1 = 5.0+50.0 = 55.0\,\text{s}$; saving $= 500.0-55.0 = 445.0\,\text{s}$; ratio $500.0/55.0 = 9.1$. ✓ The fixed 5 s warm-up is invisible against 500 reused calls — exactly what Equation (2) predicts once $n \gg n^\star$.

> **Intuition**
>
> The break-even says a pool is worth it the moment you make more calls than the pool is wide. With K=5 and our times, that is just under six calls — so for any real agent that loops more than a handful of times, the warm pool is a free lunch on latency. Ephemeral-per-call is the safe default for *rare* execution; warm pools are the default for *hot* paths. Same isolation strength, two latency regimes.

6 Step 5 — blast radius: lowering P vs. capping impact

Probability is only half the story. The module frames RCE, exfiltration, and SSRF as the *cost* side of risk. Risk is the product of how often a breach lands and how much one costs:

$$\boxed{\text{Risk} \;=\; P(\text{breach}) \times \text{impact}} \tag{3}$$

Defense-in-depth attacks the first factor; *scoping* attacks the second. Put illustrative dollars on a single breach — say \$100,000 of damage if untrusted code gets full reach (RCE to root, a customer table exfiltrated, an internal API hit via SSRF):

Posture	P(breach)	**impact**	**Risk**
1 layer, full reach	0.100	\$100,000	\$10,000.00
3 layers, full reach	0.001	\$100,000	\$100.00
3 layers *and* scoped	0.001	\$1,000	\$1.00

Sample check: $0.001 \times \$100{,}000 = \$100$; then scoping the sandbox (no secrets mounted, egress allowlisted, ephemeral filesystem) cuts the impact 100× to \$1,000, so $0.001 \times \$1{,}000 = \$1$. ✓ Stacking layers took risk from \$10,000 to \$100 (a 100× cut in P); scoping took it from \$100 to \$1 (a 100× cut in impact). The two levers *multiply*.

Intuition

Layers multiply your safety because *probabilities* multiply — each independent gate divides the chance of a breach. Scoping multiplies it again from the other side: even on the rare run where every gate fails, a sandbox with no credentials, no writable host disk, and no open egress simply cannot do \$100,000 of damage. Defense-in-depth lowers *how often*; scoping caps *how bad*. You want both, because Risk is their product.

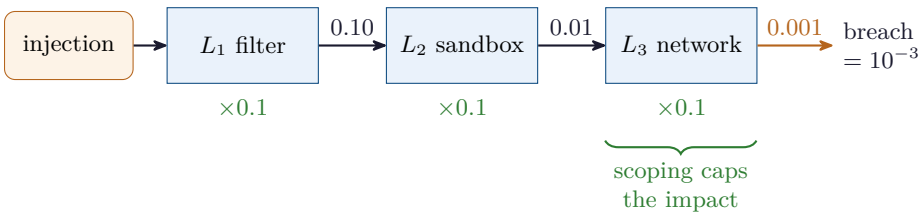
7 What changes these numbers — and what does not

Layer quality changes the base, not the rule. Replace the toy $p = 0.10$ with a strong filter at $p = 0.30$ failure and a microVM at $p = 0.01$ escape, and Equation (1) still just multiplies them. The product is always smaller than the worst single layer — that is the whole point.

Independence is the load-bearing assumption. If two layers fail for the *same* reason (e.g. both rely on the same signature database), their failures correlate and the product over-states your safety. Real stacks use deliberately *different* signals — content filtering, OS-level isolation, network egress — precisely to keep the failures independent.

What never changes: stacking independent layers turns a linear improvement in count into an exponential improvement in breach probability, and the warm-pool tax is always linear in the per-call cold start. Change the constants and the dollars and seconds move; the multiply-and-amortize *shape* does not.

8 The defense stack, drawn



Each gate lets through one-tenth of what reached it, so the surviving fraction reads $0.10 \to 0.01 \to 0.001$ across the three boxes — the running product of Equation (1). The green brace is the second lever: even the 0.1% that survives hits a scoped sandbox that caps the damage. The picture *is* Risk = P(breach) × impact.

9 Security & sandbox cheat-sheet

Breach prob., L independent layers	$P(\text{breach}) = \prod_{i=1}^{L} p_i$
Equal layers special case	$P(\text{breach}) = p^L$
Ephemeral latency tax	$\text{tax} = N\, c_{\text{cold}}$
Warm-pool latency tax	$\text{tax} = N\, c_{\text{warm}}$
Warm-pool break-even calls	$n^\star = \dfrac{K\, c_{\text{cold}}}{c_{\text{cold}} - c_{\text{warm}}}$
Pooled total latency over n calls	$K\, c_{\text{cold}} + n\, c_{\text{warm}}$
Risk	$\text{Risk} = P(\text{breach}) \times \text{impact}$
Two levers	layers $\downarrow P$ $\quad\cdot\quad$ scoping $\downarrow$ impact

10 An honest word about these numbers

The per-layer failure rate ($p = 0.10$), the cold-start times (1.0 s and 0.1 s), the pool size ($K = 5$), and the \$100,000 impact are illustrative round numbers, picked so every product and break-even is followable on paper — exactly as the ReAct-cost document used round token sizes no one would ship. Your real p_i depend on how good each control actually is on adversarial input (a strong microVM escapes far below 0.10; an injection filter on novel attacks may sit well above it), your cold starts depend on the vendor (E2B's ∼100 ms vs. a heavier roll-your-own), and impact depends entirely on what the sandbox can touch. What is *not* illustrative is the structure: independent layers multiply, so breach probability falls exponentially with depth; the latency tax is linear in the per-call cold start; and risk is always probability times impact, so defense-in-depth and scoping are two independent multipliers you apply together. Change the constants and the figures move; the shape does not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the defense-in-depth table, the latency tax, the break-even pool size, and the risk table; change `p`, `c_cold`, `c_warm`, `K`, or `impact` to explore your own stack.

```python
# sandbox_defense.py -- reproduces every number in
# "Sandbox Overhead and Defense-in-Depth".

# ---- Defense-in-depth: P(breach) = product of per-layer failure probs ----
def p_breach(p, L): # L identical independent layers, each fails w.p. p
    return p ** L

print("=== Defense-in-depth, each layer p=0.10 ===")
prev = None
for L in range(1, 6):
    pb = p_breach(0.10, L)
    rel = 0.10 / pb # reduction vs a single layer
    drop = "" if prev is None else f"{prev - pb:.5f}"
    print(f"L={L} P(breach)={pb:.6f} rel={rel:7.0f}x abs_drop={drop}")
    prev = pb

# ---- Three independent layers, the worked case ----
ps = [0.10, 0.10, 0.10]
```

```python
prod = 1.0
for p in ps:
    prod *= p
print(f"\n3 layers {ps} -> P(breach)={prod:.4f} "
      f"(single leaves {0.10:.0%}, three leave {prod:.1%}, "
      f"{0.10/prod:.0f}x reduction)")

# ---- Sandbox overhead: ephemeral vs warm pool ----
c_cold, c_warm, N = 1.0, 0.1, 5
eph, warm = N * c_cold, N * c_warm
print(f"\n=== Sandbox tax over N={N} calls ===")
print(f"ephemeral={eph:.1f}s warm={warm:.1f}s "
      f"saved={eph-warm:.1f}s ratio={eph/warm:.0f}x")

# ---- Break-even pool size ----
K = 5
n_star = K * c_cold / (c_cold - c_warm)
print(f"\nbreak-even calls n* = {K}*{c_cold}/({c_cold}-{c_warm}) = {n_star:.2f}")
M = 100 # trajectories of N calls each
no_pool = M * N * c_cold
pool = K * c_cold + M * N * c_warm
print(f"over M={M} trajectories ({M*N} calls): "
      f"no_pool={no_pool:.1f}s pool={pool:.1f}s "
      f"saved={no_pool-pool:.1f}s ratio={no_pool/pool:.1f}x")

# ---- Blast radius: Risk = P(breach) * impact ----
impact = 100_000
print("\n=== Risk = P(breach) * impact ===")
for L in (1, 3):
    pb = p_breach(0.10, L)
    print(f"L={L}, full reach : {pb:.3f} x ${impact:,} = ${pb*impact:,.2f}")
scoped = 1_000
pb3 = p_breach(0.10, 3)
print(f"L=3 + scoped : {pb3:.3f} x ${scoped:,} = ${pb3*scoped:,.2f}")

# Expected output:
# === Defense-in-depth, each layer p=0.10 ===
# L=1 P(breach)=0.100000 rel= 1x abs_drop=
# L=2 P(breach)=0.010000 rel= 10x abs_drop=0.09000
# L=3 P(breach)=0.001000 rel= 100x abs_drop=0.00900
# L=4 P(breach)=0.000100 rel= 1000x abs_drop=0.00090
# L=5 P(breach)=0.000010 rel= 10000x abs_drop=0.00009
#
# 3 layers [0.1, 0.1, 0.1] -> P(breach)=0.0010 (single leaves 10%, three leave 0.1%, 100x
    reduction)
#
# === Sandbox tax over N=5 calls ===
# ephemeral=5.0s warm=0.5s saved=4.5s ratio=10x
#
# break-even calls n* = 5*1.0/(1.0-0.1) = 5.56
# over M=100 trajectories (500 calls): no_pool=500.0s pool=55.0s saved=445.0s ratio=9.1x
#
# === Risk = P(breach) * impact ===
# L=1, full reach : 0.100 x $100,000 = $10,000.00
# L=3, full reach : 0.001 x $100,000 = $100.00
# L=3 + scoped : 0.001 x $1,000 = $1.00
```

— end of document —